

הרחבה מפורטת: שלבים ומסמכים, בדיקות ואימות, תכן וקידוד

יוצא מ-21:38:18, 1.10.2026 • Claude

CLAUDE

בקצרה: כאן מורחבים שלושת החלקים שבחרת. בכל חלק יש את המבנה הפנימי (מה יש בכל מסמך, סעיף אחרי סעיף), טכניקות מעשיות עם דוגמאות, ומה בודקים בסקרים.

חלק א': שלבים ומסמכים

1. תוכנית פיתוח התוכנה – SDP (Software Development Plan)

מה זה:

המסמך שמגדיר **איך** הפרויקט יתנהל, מהתחלה ועד המסירה. זה "חוזה העבודה" של צוות הפיתוח. כל שאר המסמכים נכתבים לפי מה שהוגדר בו.

מתי נכתב:

בשלב הייזום, לפני שמתחילים לפתח. מתעדכן לאורך הפרויקט כשמשו משתנה.

מבנה עיקרי:

סעיף	מה כולל
היקף	מה המערכת, מה ה-CSCI-ים, מי הלקוח
מסמכים ישימים	תקנים ומסמכים שהפרויקט מחויב אליהם
סקירת העבודה	מה נדרש לבצע ומה האילוצים
תהליך הפיתוח	המודל (Waterfall, V, Agile, היברידי) ושלביו
תוכניות כלליות	שימוש חוזר בקוד, טיפול בדרישות קריטיות (בטיחות, סייבר), ניצול משאבי חומרה
פעילויות מפורטות	תוכנית לכל שלב: דרישות, תכן, קידוד ובדיקות יחידה, אינטגרציה, בדיקות אישור
ניהול תצורה	הכלים, שיטת הענפים, איך מסמנים גרסאות
אבטחת איכות	איך מוודאים שהתהליך מתבצע
סקירות	אילו סקרים (SRR, PDR, CDR, TRR) ומתי
סקרי קוד	על מה עושים סקר ובאיזה היקף
ניהול סיכונים	איך מזהים ומטפלים
מדדים	מה מודדים (תקלות, כיסוי, התקדמות)
סביבת פיתוח	מחשבים, מערכות הפעלה, קומפילרים, כלים
לוחות זמנים	אבני דרך ותלויות
ארגון	תפקידים, אחריות ומשאבים

למה הוא חשוב בפועל:

כשעולה שאלה כמו "האם צריך לעשות סקר על השינוי הזה?" או "באיזה כלי מנהלים את הבינאריים?", התשובה נמצאת ב-SDP. אם עשית משהו שלא לפי ה-SDP, זו חריגה שצריך לתעד.

2. מפרט דרישות התוכנה – SRS (Software Requirements Specification)

מה זה:

המסמך שמגדיר מה התוכנה צריכה לעשות. זה הבסיס לתכן ולבדיקות, ולכן איכות הדרישות קובעת את איכות כל הפרויקט.

מבנה עיקרי לפי MIL-STD-498:

סעיף	מה כולל
היקף ומסמכים ישימים	על איזה CSCI מדובר
מצבים ואופנים	באילו מצבים המערכת עובדת (אתחול, פעולה רגילה, תחזוקה, תקלה)
דרישות יכולת	הדרישות הפונקציונליות עצמן
ממשקים חיצוניים	מול מערכות אחרות (בהפניה ל-IRS/ICD)
ממשקים פנימיים	בין רכיבי ה-CSCI
נתונים פנימיים	מבני נתונים שהתוכנה מחזיקה
דרישות התאמה	פרמטרים שמשתנים בין התקנות
דרישות בטיחות	מה אסור שיקרה, ואיך מונעים
דרישות אבטחה ופרטיות	הצפנה, הרשאות, הגנה על מידע
סביבת ה-CSCI	על איזו חומרה ומערכת הפעלה רץ
משאבי מחשב	זיכרון, מעבד, אחסון
גורמי איכות	אמינות, תחזוקתיות, זמינות, ניידות
אילוץי תכן ומימוש	שפה, תקני קידוד, ספריות מחייבות
הוראות אימות	לכל דרישה: שיטת אימות (D/T/A/I)
עקיבות	לכל דרישה: מאיזו דרישת מערכת (SSS) היא נגזרה

מה הופך דרישה לטובה

Unambiguous – חד-משמעית

יש לה פירוש אחד בלבד.

Verifiable / Testable – ניתנת לבדיקה

אפשר להוכיח שהיא מתקיימת.

Atomic – אטומית

דרישה אחת בכל משפט. לא "המערכת תשמור ותציג ותשלח".

Complete – שלמה

לא חסר מידע כדי לממש אותה.

Consistent – עקבית

לא סותרת דרישה אחרת.

Feasible – ניתנת למימוש

אפשרית טכנית ובמסגרת המשאבים.

ניתנת לעקיבה – Traceable

יש לה מזהה ייחודי ומקור.

נחוצה – Necessary

יש סיבה אמיתית שהיא קיימת.

דוגמאות: דרישה רעה מול טובה

דרישה רעה	הבעיה	דרישה טובה
"המערכת תהיה מהירה"	לא ניתנת לבדיקה	"SRS-042: המערכת תחזיר תשובה לשאילתה תוך 200ms לכל היותר, ב-95% מהמקרים"
"המערכת תתמוך במשתמשים רבים"	עמומה	"SRS-043: המערכת תתמוך ב-50 משתמשים מחוברים בו-זמנית"
"המערכת תשמור את הנתונים ותציג אותם"	לא אטומית	שתי דרישות נפרדות: שמירה, והצגה
"המערכת תהיה ידידותית"	סובייקטיבית	"SRS-044: כל פעולה תושג ב-3 לחיצות לכל היותר מהמסך הראשי"

המילה "shall":

במסמכים באנגלית, "shall" מסמנת דרישה מחייבת, "should" מסמנת המלצה, ו-"will" מסמנת עובדה. בעברית משתמשים ב"המערכת תבצע".

3. תיאור תכן התוכנה – SDD (Software Design Description)

מה זה:

המסמך שמתאר איך ה-CSCI בנוי ואיך הוא מממש את הדרישות.

מבנה עיקרי לפי MIL-STD-498:

סעיף	מה כולל
היקף ומסמכים ישימים	על איזה CSCI מדובר
החלטות תכן כלליות	החלטות שחלות על כל ה-CSCI: שפה, ארכיטקטורה, טיפול בשגיאות, לוגים, והנימוק לכל אחת
ארכיטקטורה: רכיבים	הפירוק לרכיבים (CSC), ותפקיד כל אחד
ארכיטקטורה: מושג הביצוע	איך הרכיבים פועלים יחד בזמן ריצה: תהליכים, תהליכונים, זרימת בקרה, מצבים
ארכיטקטורה: ממשקים	הממשקים בין הרכיבים ומול מערכות חיצוניות
תכן מפורט	לכל יחידה (CSU): אלגוריתם, מבני נתונים, קלט ופלט, טיפול בשגיאות
עקיבות	איזו דרישה ב-SRS ממומשת באיזה רכיב

הנקודה החשובה:
ה-SDD לא רק מתאר את הפתרון שנבחר, אלא גם **למה** הוא נבחר (Design Rationale). בסקר CDR ישאלו "למה ככה ולא אחרת?", והתשובה צריכה להיות כתובה.

4. תיאור בדיקות התוכנה – STD (Software Test Description)

מה זה:
המסמך שמכיל את תסריטי הבדיקה המפורטים. כל בודק צריך להיות מסוגל לבצע את הבדיקה רק מקריאת המסמך, בלי לשאול את מי שכתב אותו.

מבנה עיקרי:

סעיף	מה כולל
היקף ומסמכים ישימים	על איזה CSCI ואיזו רמת בדיקה
הכנות לבדיקה	חומרה, תוכנה, צב"ד, נתונים, ואיך מכינים את הסביבה
תיאורי בדיקות	תסריט לכל בדיקה (פירוט בטבלה הבאה)
עקיבות	איזו בדיקה מכסה איזו דרישה

מבנה תסריט בדיקה בודד

שדה	מה כותבים
מזהה	STD-017
דרישות מכוסות	SRS-042, SRS-043
מטרה	מה הבדיקה מוכיחה
תנאים מקדימים	מצב המערכת לפני ההתחלה
נתוני קלט	הערכים המדויקים
צעדים	רשימה ממוספרת של פעולות
תוצאה צפויה	לכל צעד, מה אמור לקרות
קריטריון הצלחה	מתי נחשב "עבר"
הנחות ואילוצים	מה מניחים על הסביבה

דוגמה לתסריט

מזהה: STD-017

דרישה: SRS-042 (זמן תגובה עד 200ms ב-95% מהמקרים)

תנאים מקדימים: המערכת במצב פעולה רגילה, מסד הנתונים טעון ב-10,000 רשומות.

צב"ד: מחולל עומסים, כלי מדידת זמנים.

צעד	פעולה	תוצאה צפויה
1	הפעלת מחולל העומסים עם 1,000 שאילתות	המחולל מתחיל לשלוח
2	המתנה לסיום	כל 1,000 השאילתות קיבלו תשובה
3	חישוב אחוזון 95 של זמני התגובה	ערך של 200ms או פחות

קריטריון הצלחה: אחוזון 95 קטן או שווה ל-200ms.

5. דוח בדיקות התוכנה – STR (Software Test Report)

מה זה:

התיעוד של מה שקרה בפועל בהרצת הבדיקות. על סמך ה-STR מחליטים אם הגרסה מוכנה.

מבנה עיקרי:

סעיף	מה כולל
היקף	איזו גרסה נבדקה (מזהה מדויק מכלי התצורה)
סיכום כללי	הערכה כוללת של התוכנה, והמלצה
השפעת הסביבה	הבדלים בין סביבת הבדיקה לסביבה המבצעית, ומה המשמעות
תוצאות מפורטות	לכל בדיקה: עבר או נכשל, בעיות שהתגלו
חריגות	איפה הבדיקה בוצעה אחרת ממה שכתוב ב-STD, ולמה
יומן בדיקות	תיעוד כרונולוגי: מי, מתי, מה
תקלות	רשימת התקלות שנפתחו, עם מזהים

מדדים שבדרך כלל מופיעים:

- כמה בדיקות תוכננו, בוצעו, עברו ונכשלו.
- כמה תקלות נפתחו, לפי חומרה.
- אחוז כיסוי הדרישות.
- תקלות פתוחות שיוצאות עם הגרסה.

חשוב:

אם בדיקה בוצעה אחרת ממה שכתוב ב-STD (למשל צעד דולג כי הצב"ד לא היה זמין), **חובה לתעד את זה** כחריגה. STR שנראה מושלם אבל לא משקף מה קרה בפועל הוא בעיה חמורה.

6. תיאור גרסת התוכנה – SVD (Software Version Description)

מה זה:

המסמך שמלווה כל גרסה שנמסרת, ומאפשר לדעת בדיוק מה נמסר ולשחזר את זה.

מבנה עיקרי:

סעיף	מה כולל
מלאי החומרים	כל מה שנמסר פיזית: מדיה, מסמכים, רישיונות
תכולת התוכנה	כל הקבצים, הגרסאות שלהם וסכומי בקורת
שינויים שהוכנסו	כל השינויים מהגרסה הקודמת, עם הפניה למספרי תקלות ו-CR
נתוני התאמה	פרמטרים ייחודיים להתקנה הזו
מסמכים קשורים	אילו גרסאות של SRS, SDD, STD, STR תקפות לגרסה הזו
הוראות התקנה	צעד אחרי צעד
בעיות ידועות	תקלות פתוחות, השפעתן, ועקיפות (Workarounds)

דוגמה לסעיף השינויים

מזהה	סוג	תיאור
BUG-1182	תיקון	תוקנה קריסה בטעינת קובץ תצורה ריק
BUG-1190	תיקון	תוקן זמן תגובה חורג במצב עומס
CR-045	יכולת חדשה	נוספה תמיכה בייצוא דוחות ל-CSV

7. עקיבות בין המסמכים

זה הקשר שמחזיק את כל המסמכים יחד:

SSS (דרישת מערכת)

↓ נגזרת ל

SRS-042 (דרישת תוכנה)

↓ נבדקת ב

SDD: רכיב QueryHandler

↓ תוצאה

Commit a3f9c12

ממומשת ב

STD-017

קוד

עבר STR:

נמסר ב

גרסה SVD 2.1

בכל שלב שואלים שתי שאלות:

- קדימה: האם לכל דרישה יש מימוש ובדיקה?
- אחורה: האם לכל רכיב ולכל בדיקה יש דרישה שמצדיקה אותם? (רכיב בלי דרישה הוא קוד מיותר או דרישה שחסרה.)

חלק ב': בדיקות, כיסוי ושיטות אימות

1. בדיקות סטטיות ודינמיות

בדיקות סטטיות – Static Testing

בדיקה **בלי להריץ** את הקוד: סקרי קוד, סקרי מסמכים, ניתוח סטטי בכלי אוטומטי. מוצאת בעיות מוקדם וזול.

בדיקות דינמיות – Dynamic Testing

בדיקה **תוך הרצה** של הקוד: בדיקות יחידה, אינטגרציה, מערכת.

2. קופסה שחורה, לבנה ואפורה

קופסה שחורה – Black Box

בודקים לפי הדרישות, **בלי לראות את הקוד**. שואלים: "האם הפלט נכון לקלט הזה?"

קופסה לבנה – White Box

בודקים לפי **מבנה הקוד**. שואלים: "האם כל ענף בקוד רץ?"

קופסה אפורה – Grey Box

שילוב: בודקים דרך הממשק החיצוני, אבל עם ידע על המבנה הפנימי (למשל, יודעים שיש Cache ובודקים במיוחד את המקרה שהוא מתמלא).

3. טכניקות קופסה שחורה

חלוקה למחלקות שקילות – Equivalence Partitioning

מחלקים את כל הקלטים האפשריים לקבוצות שהמערכת אמורה להתייחס אליהן אותו דבר, ובודקים **נציג אחד** מכל קבוצה.

דוגמה: שדה גיל שמקבל ערכים בין 18 ל-65.

מחלקה	טווח	נציג לבדיקה
לא תקין, נמוך	מתחת ל-18	10
תקין	18 עד 65	40
לא תקין, גבוה	מעל 65	80
לא תקין, סוג	לא מספר	"abc"

ניתוח ערכי גבול – Boundary Value Analysis

רוב הבאגים נמצאים **בגבולות** (טעויות של < במקום <=). בודקים את הגבול עצמו ואת הערכים הצמודים אליו.

אותה דוגמה, גיל 18 עד 65:

ערך	צפוי
17	נדחה
18	מתקבל
19	מתקבל
64	מתקבל
65	מתקבל
66	נדחה

טבלת החלטות – Decision Table

כשהתוצאה תלויה בצירוף של כמה תנאים, בונים טבלה של כל הצירופים.
דוגמה: הנחה ניתנת אם הלקוח רשום וגם הקנייה מעל 100 ש"ח.

לקוח רשום	קנייה מעל 100	הנחה?
כן	כן	כן
כן	לא	לא
לא	כן	לא
לא	לא	לא

מעברי מצבים – State Transition Testing

כשהמערכת מתנהגת כמכונת מצבים, בודקים כל מצב, כל מעבר חוקי, ומעברים לא חוקיים.
דוגמה: Idle ← Active ← Error

ממצב	אירוע	למצב	חוקי?
Idle	Start	Active	כן
Active	Failure	Error	כן
Error	Reset	Idle	כן
Idle	Failure	?	לא — צריך לוודא שהמערכת מטפלת בזה נכון

בדיקות לפי תרחישי שימוש – Use Case Testing

בודקים תרחישים מלאים מקצה לקצה, כמו שמשמש אמיתי עובר: הזדהות, ביצוע פעולה, קבלת תוצאה, יציאה.

4. טכניקות קופסה לבנה: כיסוי קוד עם דוגמה

נשתמש בפונקציה אחת כדי להבין את ההבדל בין רמות הכיסוי:

```
def classify(a: int, b: int) -> str:
    result = "low"
    # Two conditions in one decision
    if a > 0 and b > 0:
        result = "high"
    return result
```

כיסוי שורות – Statement Coverage

כל שורה רצה לפחות פעם אחת.

בדיקה	a	b	תוצאה
1	1	1	high

בדיקה אחת מספיקה ל-100% כיסוי שורות, כי כל השורות רצו. אבל לא בדקנו בכלל מה קורה כשהתנאי שקרי.

כיסוי ענפים – Branch / Decision Coverage

כל החלטה קיבלה גם אמת וגם שקר.

בדיקה	a	b	ההחלטה
1	1	1	אמת
2	0	1	שקר

שתי בדיקות. אבל לא בדקנו אף פעם ש-b לבדו משפיע. אם בטעות מישהו היה כותב `if a > 0:` בלבד, שתי הבדיקות האלה עדיין היו עוברות.

כיסוי תנאים – Condition Coverage

כל תנאי בודד ($a > 0$, $b > 0$) קיבל גם אמת וגם שקר.

כיסוי תנאים ממוזג – MC/DC

כל תנאי בודד הוכח כמשפיע לבד על התוצאה, כשהאחרים קבועים.

בדיקה	a	b	תוצאה	מה מוכיחה
1	1	1	high	בסיס
2	0	1	low	רק a השתנה, והתוצאה השתנתה $\leftarrow$ a משפיע
3	1	0	low	רק b השתנה, והתוצאה השתנתה $\leftarrow$ b משפיע

כלל אצבע: ב-MC/DC צריך בדרך כלל $N+1$ בדיקות, כש-N הוא מספר התנאים. זה נדרש בתוכנה בטיחותית ברמה הגבוהה (DO-178C רמה A).

כיסוי מסלולים – Path Coverage

כל מסלול אפשרי בקוד מההתחלה ועד הסוף. בקוד עם לולאות מספר המסלולים יכול להיות עצום, ולכן בפועל מכסים רק מסלולים עיקריים.

סדר הרמות

מסלולים < MC/DC < שורות > ענפים > תנאים
חלש חזק

5. רמות הבדיקה לעומק

Unit Testing – בדיקות יחידה

מי: המפתח.

מה: פונקציה או מחלקה בודדת, מבודדת משאר המערכת.

איך מבודדים: מחליפים תלויות בתחליפים (Mocks, Stubs).

Integration Testing – בדיקות שילוב

מי: מפתחים או צוות אינטגרציה.

מה: שרכיבים עובדים יחד: שהממשקים תואמים, שהנתונים עוברים נכון.

אסטרטגיות שילוב:

Big Bang – הכול בבת אחת

מחברים את כל הרכיבים יחד ובודקים. פשוט, אבל כשמשהו נכשל קשה מאוד לדעת איפה.

Top-Down – למעלה למטה

מתחילים מהרכיבים העליונים (ממשק משתמש, בקרה), ומוסיפים בהדרגה את התחתונים. רכיבים תחתונים שעוד לא קיימים מוחלפים ב-Stub.

Bottom-Up – למעלה למטה

מתחילים מהרכיבים התחתונים (גישה לחומרה, מסד נתונים), ומוסיפים את העליונים. רכיבים עליונים שעוד לא קיימים מוחלפים ב-Driver.

סנדוויץ' – Sandwich / Hybrid

שילוב של שניהם במקביל.

תחליף נקרא – Stub

מחליף רכיב שהרכיב הנבדק קורא אליו. מחזיר תשובות מוכנות מראש.

תחליף קורא – Driver

מחליף רכיב שקורא לרכיב הנבדק. מפעיל אותו עם קלטים.

Top-Down:	Bottom-Up:
[רכיב נבדק]	[Driver]
↓ קורא ↓	קורא
[Stub]	[רכיב נבדק]

System Testing – מערכת – בדיקות מערכת

מי: צוות בדיקות בלתי תלוי.

מה: המערכת המלאה מול ה-SRS, כולל דרישות לא-פונקציונליות.

Acceptance Testing – קבלה – בדיקות קבלה

מי: מול הלקוח או בנוכחותו.

מה: הוכחה שהמערכת עונה על מה שהוזמן. לפי ATP מאושר מראש.

6. סוגי בדיקות לא-פונקציונליות

Performance Testing – בדיקות ביצועים

זמני תגובה ותפוקה בעומס רגיל.

Load Testing – בדיקות עומס

התנהגות בעומס הצפוי המקסימלי.

Stress Testing – בדיקות קצה

מעבר לעומס המקסימלי, כדי לראות איך המערכת נכשלת (בצורה מסודרת או בקריסה).

Endurance / Soak Testing – בדיקות סיבולת

הרצה ארוכה (שעות או ימים), כדי למצוא דליפות זיכרון ובעיות שמצטברות.

Security Testing – בדיקות אבטחה

ניסיונות חדירה, קלט זדוני, בדיקת הרשאות.

Recovery Testing – בדיקות התאוששות

מה קורה אחרי נפילת חשמל, ניתוק, או קריסת רכיב.

Usability Testing – בדיקות שמישות

כמה קל להשתמש במערכת.

7. בדיקות רגרסיה, Retest, Smoke ו-Sanity

סוג	מתי	מה בודק
בדיקה חוזרת – Retest	אחרי תיקון תקלה	שהתקלה הספציפית תוקנה
רגרסיה – Regression	אחרי כל שינוי	שלא נשבר משהו שעבד
בדיקת עשן – Smoke	כל Build חדש	שהבסיס עובד לפני בדיקות מעמיקות
בדיקת שפיות – Sanity	אחרי תיקון קטן	שהאזור הספציפי עובד

למה רגרסיה חייבת אוטומציה:

חבילת הרגרסיה גדלה עם כל גרסה. להריץ מאות תסריטים ידנית על כל שינוי זה לא מעשי, ולכן זה התחום שבו אוטומציה מחזירה את ההשקעה הכי מהר.

8. קריטריוני כניסה ויציאה

קריטריוני כניסה – Entry Criteria

מה חייב להתקיים לפני שמתחילים רמת בדיקה. למשל, לבדיקות מערכת:

- הגרסה עברה בדיקות אינטגרציה.
- הגרסה מסומנת בכלי התצורה.
- ה-STD מאושר.
- הסביבה והצב"ד מוכנים.
- אין תקלות חוסמות פתוחות.

קריטריוני יציאה – Exit Criteria

מה חייב להתקיים כדי לסיים רמת בדיקה. למשל:

- כל התסריטים בוצעו.
- 100% כיסוי דרישות.
- אין תקלות פתוחות בחומרה קריטית או גבוהה.
- ה-STR נכתב ואושר.

הקשר ל-TRR: סקר ה-TRR הוא בדיוק הבדיקה שקריטריוני הכניסה מתקיימים.

9. ניהול תקלות לעומק

חומרה מול עדיפות

חומרה – Severity

כמה חמורה ההשפעה הטכנית של התקלה.

עדיפות – Priority

כמה דחוף לתקן אותה.

חומרה	משמעות
קריטית – Critical	קריסה, איבוד נתונים, סכנה בטיחותית
גבוהה – High	פונקציה מרכזית לא עובדת, אין עקיפה
בינונית – Medium	פונקציה לא עובדת, יש עקיפה
נמוכה – Low	קוסמטי, טעות כתיב

הן לא תמיד תואמות:

- חומרה נמוכה, עדיפות גבוהה: שם החברה כתוב לא נכון במסך הראשי לפני הדגמה ללקוח.
- חומרה גבוהה, עדיפות נמוכה: קריסה בתרחיש נדיר מאוד שלא יגיע למסירה הקרובה.

מחזור החיים של תקלה



מצבים נוספים:

- **נדחתה – Rejected:** זו לא תקלה (התנהגות תקינה, או טעות בבדיקה).
- **כפולה – Duplicate:** כבר נפתחה.
- **נדחתה לגרסה הבאה – Deferred:** תקלה אמיתית, אבל לא תתוקן עכשיו. **חייבת להופיע ב-SVD כבעיה ידועה.**

דוח תקלה טוב

שדה	דוגמה
כותרת	קריסה בטעינת קובץ תצורה ריק
גרסה	2.0.3 (Build 1456)
סביבה	שרת בדיקות 2, Windows Server 2022
צעדי שחזור	1. יצירת קובץ config.json ריק. 2. הפעלת המערכת.
צפוי	הודעת שגיאה ברורה והמשך עם ברירות מחדל
בפועל	קריסה עם NullPointerException
תדירות	5 מתוך 5 ניסיונות
חומרה	גבוהה
קבצים מצורפים	לוג, צילום מסך

הכלל: מפתח שלא היה בחדר צריך להצליח לשחזר את התקלה רק מהדוח.

10. ארבע שיטות האימות לעומק

איך בוחרים שיטה

שאלה	אם כן
אפשר לראות את התוצאה בעין, בלי מדידה?	הדגמה – D
צריך מדידה מדויקת או ציוד מיוחד?	בדיקה – T
אי אפשר לבדוק ישירות (מסוכן, יקר, נדיר)?	ניתוח – A
הדרישה היא על התוצר עצמו ולא על ההתנהגות?	סקירה – I

דוגמאות לכל שיטה

הדגמה – Demonstration

- "המערכת תציג את שם המשתמש בפינת המסך."
- "לחיצה על כפתור 'שמור' תשמור את הטופס."

בדיקה – Test

- "זמן האתחול יהיה פחות מ-30 שניות." (צריך מדידה מדויקת)
- "המערכת תעבד 1,000 הודעות בשנייה." (צריך מחולל עומס)

ניתוח – Analysis

- "זמן ממוצע בין תקלות (MTBF) יהיה לפחות 10,000 שעות." (אי אפשר להריץ 10,000 שעות, מחשבים מנתונים)
- "המערכת תעמוד בעומס של 10,000 משתמשים." (בודקים ב-1,000 ומחשבים הלאה)

סקירה – Inspection

- "הקוד ייכתב לפי תקן הקידוד של הפרויקט."
- "כל ההודעות למשתמש יהיו בעברית."
- "המדריך למשתמש יכלול פרק התקנה."

שילוב שיטות:

דרישה יכולה לקבל יותר משיטה אחת. למשל, דרישת ביצועים יכולה להיות מאומתת ב**בדיקה** בעומס של 1,000, ו**ניתוח** שמראה שהתוצאה מחזיקה גם ב-10,000.

חלק ג': תכן וקידוד

1. דיאגרמות UML

דיאגרמות סטטיות (מבנה)

דיאגרמת מחלקות – Class Diagram

מחלקות, שדות, מתודות והקשרים ביניהן (ירושה, הרכבה, תלות).

דיאגרמת רכיבים – Component Diagram

הרכיבים הגדולים והממשקים ביניהם.

דיאגרמת פריסה – Deployment Diagram

על איזו חומרה רץ כל רכיב.

דיאגרמת חבילות – Package Diagram

איך הקוד מאורגן בחבילות.

דיאגרמות דינמיות (התנהגות)

דיאגרמת רצף – Sequence Diagram

סדר הקריאות בין רכיבים לאורך זמן, בתרחיש מסוים.

מכונת מצבים – State Machine Diagram

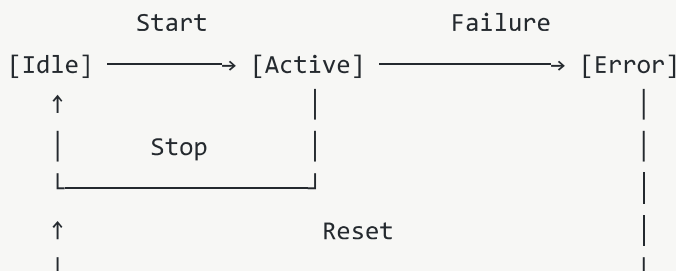
המצבים שרכיב יכול להיות בהם, והאירועים שמעבירים ביניהם.

דיאגרמת פעילות – Activity Diagram

זרימת פעולות, כמו תרשים זרימה.

דיאגרמת תרחישי שימוש – Use Case Diagram

מי משתמש במערכת ולמה.

דוגמה למכונת מצבים**מה חייב להיות מוגדר בתכן:**

- מה המצב ההתחלתי.
- לכל מצב: אילו אירועים מותרים ולכן הם מובילים.
- **מה קורה עם אירוע לא צפוי** (למשל Failure במצב Idle). אם זה לא מוגדר, זה באג שמחכה לקרות.

2. צימוד ולכידות לעומק**סוגי צימוד, מהגרוע לטוב****צימוד תוכן – Content Coupling**

רכיב ניגש ישירות לנתונים הפנימיים של רכיב אחר. הגרוע ביותר.

צימוד משותף – Common Coupling

כמה רכיבים חולקים משתנים גלובליים.

צימוד בקרה – Control Coupling

רכיב מעביר דגל שקובע מה רכיב אחר יעשה.

צימוד מבנה – Stamp Coupling

מעבירים מבנה נתונים שלם כשצריך רק חלק ממנו.

צימוד נתונים – Data Coupling

מעבירים רק את הנתונים שצריך, כפרמטרים. הטוב ביותר.

סוגי לכידות, מהגרועה לטובה**לכידות מקרית – Coincidental**

דברים לא קשורים שנמצאים יחד (כמו מחלקת Utils עם הכול).

לכידות לוגית – Logical

פעולות מאותו סוג, אבל לא קשורות (כל הפונקציות שמדפיסות).

לכידות זמנית – Temporal

דברים שקורים באותו זמן (כל מה שרץ באתחול).

לכידות פרוצדורלית – Procedural

צעדים שרצים ברצף.

לכידות תקשורתית – Communicational

פעולות על אותם נתונים.

לכידות רציפה – Sequential

הפלט של פעולה אחת הוא הקלט של הבאה.

לכידות פונקציונלית – Functional

כל הרכיב עושה דבר אחד מוגדר. הטובה ביותר.

3. עקרונות SOLID**Single Responsibility – אחריות יחידה**

לכל מחלקה סיבה אחת להשתנות.

Open-Closed – פתוח-סגור

פתוח להרחבה, סגור לשינוי. מוסיפים התנהגות חדשה על ידי מחלקה חדשה, לא על ידי שינוי קוד קיים.

Liskov Substitution – החלפה של ליסקוב

מחלקה יורשת חייבת להיות ניתנת להחלפה במקום מחלקת הבסיס בלי לשבור שום דבר.

Interface Segregation – הפרדת ממשקים

עדיף כמה ממשקים קטנים וממוקדים מאשר ממשק אחד גדול שמכריח לממש דברים לא רלוונטיים.

Dependency Inversion – היפוך תלויות

רכיבים עליונים תלויים בממשקים (הפשטות), לא במימושים ספציפיים. זה גם מה שמאפשר להחליף תלות ב-Mock בבדיקות.

דוגמה לעקרון פתוח-סגור

```

from abc import ABC, abstractmethod

class Exporter(ABC):
    @abstractmethod
    def export(self, data: list[dict]) -> str: ...

class CsvExporter(Exporter):
    def export(self, data: list[dict]) -> str:
        # Build CSV text from rows
        ...

class JsonExporter(Exporter):
    def export(self, data: list[dict]) -> str:
        # Build JSON text from rows
        ...

def save_report(data: list[dict], exporter: Exporter) -> str:
    # This function never changes when a new format is added
    return exporter.export(data)

```

כדי להוסיף ייצוא ל-XML, כותבים מחלקה חדשה `XmlExporter`. הפונקציה `save_report`, שכבר נבדקה, לא משתנה.

4. תבניות תכן – Design Patterns

תבניות יצירה – Creational

- **Singleton**: מופע יחיד של מחלקה (למשל מנהל לוגים).
- **Factory**: יצירת אובייקטים בלי לציין את המחלקה המדויקת.
- **Builder**: בניית אובייקט מורכב שלב אחרי שלב.

תבניות מבנה – Structural

- **Adapter**: מתאם בין שני ממשקים שלא תואמים.
- **Facade**: ממשק פשוט מעל מערכת מורכבת.
- **Decorator**: הוספת התנהגות לאובייקט בלי לשנות אותו.

תבניות התנהגות – Behavioral

- **Observer**: כשאובייקט משתנה, כל המנויים מקבלים הודעה.
- **Strategy**: החלפת אלגוריתם בזמן ריצה (הדוגמה של Exporter למעלה).
- **State**: התנהגות שמשתנה לפי מצב פנימי (מימוש מכונת מצבים).
- **Command**: עטיפת בקשה כאובייקט (לתור, ביטול, תיעוד).

5. עקרונות תכן נוספים

אל תחזור על עצמך – DRY (Don't Repeat Yourself)

כל ידע מוגדר במקום אחד בלבד.

שמור על פשטות – KISS (Keep It Simple)

הפתרון הפשוט ביותר שעובד.

לא תצטרך את זה – YAGNI (You Aren't Gonna Need It)

לא לבנות יכולות "למקרה שיצטרכו".

תכן לבדיקתיות – Design for Testability

לתכנן מראש כך שיהיה קל לבדוק: תלויות דרך ממשקים, פונקציות קטנות, בלי מצב גלובלי.

6. רכיבי תכן לדרישות בטיחות וסייבר**בטיחות****Watchdog – כלב שמירה**

טיימר שמאתחל את המערכת אם היא נתקעה.

Redundancy – יתירות

רכיבים כפולים, כך שנפילה של אחד לא מפילה את המערכת.

Fail-Safe – מצב כשל בטוח

כשמשהו נכשל, המערכת עוברת למצב שלא מסוכן.

BIT (Built-In Test) – בדיקות עצמיות

המערכת בודקת את עצמה באתחול ובמהלך פעולה.

Input Validation – בדיקת תקינות קלט

כל קלט נבדק לפני שימוש.

סייבר**Least Privilege – הרשאה מינימלית**

כל רכיב מקבל רק את ההרשאות שהוא צריך.

Encryption – הצפנה

של נתונים בתנועה ובמנוחה.

Authentication – אימות

וידוא זהות לפני גישה.

Secure Boot – אתחול מאובטח

רק קוד חתום יכול לרוץ.

Audit Logging – רישום אירועים

תיעוד פעולות רגישות.

7. תקני קידוד – Coding Standards**מה זה:**

כללים מחייבים לכתיבת קוד: שמות, מבנה, מה מותר ומה אסור. מוגדר ב-SDP.

תקן	שפה	תחום
++MISRA C / C	++C, C	מערכות בטיחותיות ומשובצות
SEI CERT	C, C++, Java	אבטחת קוד
++AUTOSAR C	++C	רכב
PEP 8	Python	סגנון כללי
Google Style Guides	שונות	סגנון כללי

דוגמאות לכללים נפוצים:

- אסור להשתמש במשתנים גלובליים שאינם קבועים.
- כל `switch` חייב `default`.
- אסור הקצאת זיכרון דינמית אחרי האתחול (במערכות בזמן אמת).
- חובה לבדוק ערך החזרה של כל פונקציה.

8. ניתוח סטטי – Static Analysis

מה זה:

כלי שקורא את הקוד **בלי להריץ** ומוצא בעיות: הפרות תקן, באגים אפשריים, פגיעויות.

כלי	שפה
ruff, pylint, mypy	Python
clang-tidy, cppcheck	++C, C
Coverity, Klocwork	שונות, מסחרי
SonarQube	שונות, כולל דשבורד

איפה זה משתלב: רץ אוטומטית ב-CI על כל שינוי, ולעיתים חוסם מיזוג אם יש הפרות.

9. סקר קוד לעומק

מה בודקים

נכונות – Correctness

הקוד עושה את מה שהדרישה אומרת. מקרי קצה מטופלים.

טיפול בשגיאות – Error Handling

כל שגיאה אפשרית נתפסת ומטופלת. אין שגיאות שנבלעות בשקט.

עמידה בתקן – Standards

לפי תקן הקידוד.

קריאות – Readability

שמות ברורים, פונקציות קצרות, הערות במקום שצריך.

תכן – Design

תואם ל-SDD. אין צימוד מיותר.

אבטחה – Security

בדיקת קלט, אין סיסמאות בקוד, אין הזרקות.

ביצועים – Performance

אין לולאות מיותרות, אין דליפות משאבים.

בדיקות – Tests

יש בדיקות יחידה, והן בודקות את מה שחשוב.

עקיבות – Traceability

השינוי מקושר לדרישה או לתקלה.

איך עושים סקר טוב

- 200 עד 400 שורות בסקר.
- עד 60–90 דקות.
- הערות על הקוד, לא על האדם.
- להבחין בין "חובה לתקן" ל"הצעה".
- **לתעד** את הסקר: מי סקר, מה נמצא, מה תוקן. בתעשייה הביטחונית זה חלק מאבטחת האיכות.

10. בדיקות יחידה לעומק**מבנה בדיקה: AAA****הכנה – Arrange**

מכינים את הנתונים והתנאים.

פעולה – Act

מריצים את מה שנבדק.

וידוא – Assert

בודקים את התוצאה.

```

import pytest

from calculator import divide

def test_divide_returns_quotient() -> None:
    # Arrange
    numerator, denominator = 10, 2
    # Act
    result = divide(numerator, denominator)
    # Assert
    assert result == 5

def test_divide_by_zero_raises() -> None:
    # Division by zero must raise, not return a value
    with pytest.raises(ZeroDivisionError):
        divide(1, 0)

```

עקרונות FIRST

מהירה – Fast

רצה בשניות, כדי שירוצו כל הזמן.

בלתי תלויה – Independent

לא תלויה בבדיקות אחרות או בסדר ההרצה.

חוזרת – Repeatable

אותה תוצאה בכל הרצה ובכל מחשב.

מאמתת את עצמה – Self-Validating

עוברת או נכשלת, בלי צורך לקרוא פלט.

בזמן – Timely

נכתבת יחד עם הקוד, לא חודשים אחר כך.

תחליפים לתלויות

תחליף מדמה – Mock

אובייקט מזויף שגם **מוודא** שקראו לו נכון.

תחליף פשוט – Stub

אובייקט שרק מחזיר תשובות מוכנות.

תחליף עובד – Fake

מימוש פשוט שעובד באמת (למשל מסד נתונים בזיכרון).

```

from unittest.mock import Mock

from report import send_report

def test_send_report_calls_mailer_once() -> None:
    # Replace the real mailer so no email is sent
    mailer = Mock()
    send_report(mailer, recipient="team@example.com")
    mailer.send.assert_called_once()

```

פיתוח מונחה בדיקות – TDD (Test-Driven Development)

1. **אדום:** כותבים בדיקה שנכשלת.
2. **ירוק:** כותבים את הקוד המינימלי שגורם לה לעבור.
3. **שיפור:** משפרים את הקוד (Refactor), והבדיקה ממשיכה לעבור.

11. ניהול ענפים וקומיטים

אסטרטגיות ענפים

גיט פלואו – Git Flow

ענף **main** לגרסאות משוחררות, ענף **develop** לפיתוח, ענף נפרד לכל יכולת, וענפי **release** ו- **hotfix**. מתאים לשחרורים מתוכננים.

פיתוח על הגזע – Trunk-Based Development

כולם מתמזגים לענף ראשי לעיתים קרובות, עם ענפים קצרים מאוד. מתאים ל-CI/CD מהיר.

הודעת קומיט טובה

BUG-1182: Handle empty configuration file

Return default settings instead of crashing when config.json exists but contains no data.

- מתחילה במזהה התקלה או הדרישה (עקיבות).
- שורה ראשונה קצרה שאומרת **מה**.
- גוף שמסביר **למה**.

12. צינור CI/CD

אינטגרציה רציפה – CI (Continuous Integration)

כל שינוי מתמזג ונבדק אוטומטית.

מסירה רציפה – CD (Continuous Delivery)

כל גרסה שעוברת את הצינור מוכנה לשחרור.

שלבים טיפוסיים בצינור:

שלב	מה קורה	אם נכשל
משיכה – Checkout	הקוד נמשך מכלי התצורה	–
ניתוח סטטי – Lint	ruff, clang-tidy	חוסם
בנייה – Build	קומפילציה	חוסם
בדיקות יחידה – Unit Tests	כל הבדיקות רצות	חוסם
מדידת כיסוי – Coverage	אחוז כיסוי	חוסם אם מתחת לסף
סריקת אבטחה – Security Scan	פגיעויות בתלויות	חוסם בחומרה גבוהה
שמירת תוצר – Publish	הבינארי נשמר בכלי התוצרים	–
בדיקות אינטגרציה	על סביבה מוקמת	חוסם

13. תכנות הגנתי – Defensive Programming

מה זה:

כתיבת קוד שמניח שדברים ישתבשו, ומטפל בזה.

עקרונות:

- לבדוק כל קלט, גם מרכיבים "פנימיים".
- לבדוק ערכי החזרה.
- להשתמש בטענות (Assertions) לתנאים שלעולם לא אמורים לקרות.
- לא לבלוע שגיאות בשקט. לפחות לרשום בלוג.
- לשחרר משאבים גם במקרה של שגיאה (`with` ב-Python, RAII ב-C++).

```
import logging

logger = logging.getLogger(__name__)

def read_sensor_value(raw: str) -> float | None:
    # Reject input that cannot represent a valid reading
    try:
        value = float(raw)
    except ValueError:
        logger.warning("Invalid sensor reading: %r", raw)
        return None
    if not -50.0 <= value <= 150.0:
        logger.warning("Sensor reading out of range: %s", value)
        return None
    return value
```

סיכום: שאלות לחזרה על ההרחבה

שאלה	תשובה
מה מגדיר ה-SDP לגבי סקרי קוד?	על אילו חלקים עושים סקר ובאיזה היקף
מה חייב להיות לכל דרישה ב-SRS?	מזהה, ניסוח חד-משמעי, שיטת אימות, מקור
מה ההבדל בין סעיף הארכיטקטורה לסעיף התכנה המפורט ב-SDD?	ארכיטקטורה: רכיבים ויחסים. מפורט: אלגוריתמים ומבני נתונים של כל יחידה
מה חייב להופיע ב-STR אם בדיקה בוצעה אחרת מהמתוכנן?	תיעוד כחריגה, עם הסבר
מה ההבדל בין חלוקה לשקילות לבין ערכי גבול?	שקילות: נציג מכל קבוצה. גבול: הערכים בקצה הקבוצות
למה כיסוי שורות הוא החלש?	אפשר 100% בלי לבדוק את המקרה השקרי
כמה בדיקות צריך ל-MC/DC?	בדרך כלל N+1, כש-N מספר התנאים
מה ההבדל בין Stub ל-Driver?	Stub מחליף רכיב שקוראים לו. Driver מחליף רכיב שקורא
מה ההבדל בין חומרה לעדיפות?	חומרה: כמה חמור. עדיפות: כמה דחוף
מה קורה לתקלה שנדחתה לגרסה הבאה?	חייבת להופיע ב-SVD כבעיה ידועה
איזה צימוד הכי טוב?	צימוד נתונים
איזו לכידות הכי טובה?	לכידות פונקציונלית
מה מאפשר להחליף תלות ב-Mock?	היפוך תלויות (Dependency Inversion)
מה זה AAA?	Arrange, Act, Assert
למה הודעת קומיט מתחילה במזהה?	עקיבות מהקוד לדרישה או לתקלה